Polynomial Interpolation Strategies

Jonathan Cushman

March 2026

1 Executive Summary

This report evaluates the numerical behavior associated with the three interpolating forms: Barycentric-1, Barycentric-2, and Newton Interpolation Form. The methods are assessed using various mesh types: Uniform, Chebyshev of the first kind, and Chebyshev of the second kind. Along with various orderings: Increasing, Decreasing, and Leja's ordering for testing numerical robustness.

Key Findings:

- Conditioning: A uniform mesh produces a much higher Lebesgue Constants Λ_n in comparison to the Chebyshev meshes. Thus leading to numerical instability as the degree n increases.
- Stability: The Barycentric-2 Form exhibits extreme numerical stability, especially when paired with Chebyshev nodes, which matches the findings from the Berrut and Trefethen Paper.
- Precision: Throughout the testing and validation, single point precision helps to reveal the drawbacks of floating point arithmetic when dealing with an ill-conditioned problem. Therefore, double point precision is used as the "exact" value used for the error analysis.
- Ordering: For Newton's Interpolating Form, the Leja's ordering helps to provide more stability than increasing or decreasing orders. This helps to protect against potential overflow issues that can occur in the divided difference computations.

2 Statement of the Problem

The main objective of this assignment is the empirical evaluation of polynomial interpolation which shows how the choice of basis, mesh distribution, and computed form affects the accuracy and stability of the polynomial $p_n(x)$.

The problem can be segmented into the following components:

- Algorithmic Implementation: Design and implementation of routines that generate coefficients and evaluate the polynomials for the various interpolating forms in $O(n)$ and $O(n^2)$.
- Conditioning Analysis: Empirically evaluate the sensitivity of the interpolation through the conditioning numbers: $\kappa(x, n, y)$ and $\kappa(x, n, 1)$. Evaluating the growth of Λ_n across the different meshes.
- Numerical Stability: Comparison in performance between single and double point precision. Specifically looking at how rounding errors propagate through different mathematical descriptions of the same polynomial.
- Convergence: Investigating the convergence behavior for the Runge function $f_4(x)$ to determine the threshold for which the degree n no longer improves the approximation, forced by the numerical instability.

The Analysis is conducted using 4 test functions, providing a rigorous testing environment for the Algorithms.

3 Description of the Mathematics

This section is an outline of the different frameworks that the design and implementation of the routines are based upon. Focusing on the 3 representations of the interpolating polynomial $p_n(x)$ that connects the data points (x_i, y_i) for $i = 0, ..., n$.

Start by defining the Barycentric-1 interpolation formula for $0 \leq i \leq n$

$$p_n(x) = \omega_{n+1}(x) \sum_{i=0}^{n} y_i \frac{\gamma_i}{(x - x_i)}, \ \gamma_i = \frac{1}{\omega'_{n+1}(x_i)}, \ \omega_{n+1}(x) = \prod_{i=0}^{n} (x - x_i) \quad (1)$$

From this definition, construction of $p_n(x)$ is independent of y_i but requires the computation of the weights γ_i. Construction of the polynomial can be done in $O(n^2)$ computations and $O(n)$ storage.

Next, define the formula for Barycentric-2 interpolation given distinct x_i from $0 \leq i \leq n$

$$p_n(x) = \frac{\sum_{i=0}^{n} y_i \frac{\beta_i}{(x - x_i)}}{\sum_{i=0}^{n} \frac{\beta_i}{(x - x_i)}}, \quad (2)$$

Once again, construction of $p_n(x)$ does not depend on y_i. This form is often preferred for computational performance due to the numerical stability. The weights β_i are independent of the function values $f(x_i) = y_i$ and only depend upon the mesh. This means they can be computed once for specific meshes like Chebyshev points and reused for any function interpolated on that same mesh.

The Barycentric weights β_i are constructed differently dependent upon the choice of mesh used. The distribution of the mesh points x_i is a primary factor in the conditioning of the interpolation problem.

- If the interpolation nodes are given be a uniform mesh: $O(n)$

$$x_i = x + ih \implies \beta_i = (-1)^i \binom{n}{i} \tag{3}$$

- If the interpolation nodes are Chebyshev of the first kind: $O(n)$

$$x_i = cos(\frac{(2i+1)\pi}{2n+2}) \implies \beta_i = (-1)^i sin(\frac{(2i+1)\pi}{2n+2}) \tag{4}$$

- if the interpolation nodes are Chebyshev of the second kind: $O(1)$

$$x_i = cos(\frac{i\pi}{n}) \implies \beta_i = (-1)^i \delta_i,\ \delta_i = \begin{cases} 1/2 & \text{if } i = 0 \text{ or } i = n \\ 1 & \text{Otherwise} \end{cases} \tag{5}$$

Next, define the formula for Newton's interpolating form:

$$p_n(x) = \sum_{i=0}^{n} y[x_0, ..., x_i]\omega_i(x),\ \omega_i(x) = \prod_{i=0}^{i-1}(x - x_i) \tag{6}$$

Which given the basis choice and the required divided differences, $p_n(x)$ is evaluated in $O(n)$ operations and space using the generalized algorithm of Horner's Rule to construct the polynomial.

To measure the sensitivity of the interpolating problem, define the following condition numbers:

$$\kappa(x, n, y) = \frac{\sum_{i=0}^{n} |l_i(x)y_i|}{|p_n(x)|}, \quad \kappa(x, n, 1) = \sum_{i=0}^{n} |l_i(x)| \tag{7}$$

$$H_n = max_{x\in[a,b]}\kappa(x, n, y), \quad \Lambda_n = max_{x\in[a,b]}\kappa(x, n, 1) \tag{8}$$

Where $\kappa(x, n, y)$ is the relative conditioning number that evaluates how errors are propagated throughout the entire interpolation process. The second conditioning number is the Lebesgue Constant Λ_n, which measures the sensitivity of the problem with respect to perturbations in the function values y_i.

The last thing to consider is the ordering of the nodes for Newton's interpolating form.

- Increasing: $x_0 < x_1 < x_2 < ... < x_n$
- Decreasing: $x_0 > x_1 > x_2 > ... > x_n$

- Leja's Ordering: $\prod_{k=0}^{j-1}(x - x_k)$ (Maximize distance from previous node)

These orderings are helpful for testing the numerical stability associated with the Newton interpolating form. In single precision the ordering can have an affect on the rounding errors produced, leading vastly different resulting polynomials. Leja's ordering is especially helpful when evaluating Newton's Form. This comes from the effective spacing within the ordering that helps to improve stability when a mesh is to clustered: ie) $|x_i - x_{i+1}| \leq \epsilon$ for a high degree polynomial interpolation.

4 Task 1: Algorithm Design and Complexity

As specified in the assignment tasks, the routines are designed to run in single point precision when conducting stability testing and double point precision for generating exact values as a reference. The implementation of each routine takes in the precision flag and casts the variable accordingly.

4.1 Barycentric-1 Coefficient Generation

The Barycentric Form 1 is a reformulated version of the Lagrange interpolating polynomial that is designed to improve the computational efficiency. The primary goal of this routine is to return the weights/coefficients γ_i along with the evaluated function $f_i = f(x_i)$. The parameters for the function take in the function f, the mesh points, and the desired precision. The algorithm uses a nested loop to calculate the product for each of the $n+1$ coefficients. The algorithm must preform n subtractions and $n-1$ multiplications, resulting in $O(n^2)$ operations for the $n+1$ weights. The weights are then stored within a vector using $O(n)$ space

```
function[gamma,fx] = Baryc1Coeff(mesh,f,precision)

n = length(mesh) - 1;
mesh = double(mesh);
gamma = ones(1,n+1);

for j = 1:n+1
    for k = 1:n+1
        if j ~= k
            gamma(j) = gamma(j)*(mesh(j)-mesh(k));
        end
    end
end
gamma = 1 ./ gamma;

fx = cast(f(mesh), precision);
gamma = cast(gamma, precision);
```

```
end
```

4.2 Barycentric-1 Polynomial Evaluation

Using the previously generated γ_i and f_i values, the next routine is designed to evaluate the interpolating polynomial $p_n(x) = \omega_{n+1}(x)\sum_{i=0}^{n} y_i \frac{\gamma_i}{(x-x_i)}$. When computing the product form ω_{n+1} it requires $n+1$ subtractions and n multiplications. Then the summation that is left in our evaluation has $n+1$ terms that each require one subtraction, one multiplication, and one division. Combining all of this together results in $O(n^2)$ complexity per evaluation point.

Additionally, within the Barycentric-1 interpolating function is a conditional check for situations where $x = x_i$. This causes the term $\frac{1}{x-x_i}$ to blow up in magnitude. The check "find(difference == 0,1)" identifies the points of zero difference and returns f_i to keep the code robust.

```
function px = Bary1_Inter(eval_pts,mesh,gamma,fx,precision)

if precision == "single"
    evaluation_points = single(evaluation_points);
    mesh = single(mesh);
    gamma = single(gamma);
    fx = single(fx);
else
    mesh = double(mesh);
    evaluation_points = double(evaluation_points);
    gamma = double(gamma);
    fx = double(fx);
end

px = cast(zeros(size(evaluation_points)), precision);

for j=1:length(evaluation_points)
    x_current = evaluation_points(j);
    difference = x_current - mesh;

    exact_spot = find(difference == 0,1);

    if ~isempty(exact_spot)
        px(j) = fx(exact_spot);
    else
        lx = prod(difference);
        summation = sum((gamma .* fx ./ difference));
        px(j) = lx * summation;
    end
end
```

```
end
```

4.3 Barycentric-2 Coefficient Generation

This routine computes the weights β_i and function values f_i. One key advantage that the $2nd$ form has is that for specific meshes, the β_i can be computed without the product form required by Barycentric-1. This is useful for lowering the complexity down from $O(n^2)$.

For a uniform mesh, the weights are $\beta_j = (-1)^j \binom{n}{j}$. The algorithm sets an index $k = 0, ..., n$ and loops through to compute the binomial coefficients. For Chebyshev points of the first kind the coefficients are now $\beta_j = (-1)^j sin(\frac{(2j+1)\pi}{2n+2}$. They are evaluated under the same same conditional loop as before and thus has $n+1$ trigonometric evaluations. The last mesh is for Chebyshev points of the second kind where $\beta_j = (-1)^j \delta_j,$. This is the most efficient of the three distributions as the weights follow and alternating pattern and requires simple arithmetic. Overall, each method is reduced from $O(n^2)$ to $O(n)$ space for each distribution. This results in the routine having an overall complexity of $O(n)$ space from only storing the vectors for the mesh, weights, and function values.

```
function[beta,fx,x_mesh] = Baryc2Coeff(Interval,f,type,n,prec.)
a = Interval(1);
b = Interval(2);
k = 0:n;

if mesh_type == "uniform"
    x = -1 + (2*k) / n;
elseif mesh_type == "chebychev1"
    x = cos((2*k + 1) * pi / (2*n +2));
elseif mesh_type == "chebychev2"
    x = cos(k * pi / n);
end

x_mesh = 0.5*(a+b) + 0.5*(b-a) * x;
x_mesh = cast(x_mesh, precision);
fx = cast(f(x_mesh), precision);

if mesh_type == "uniform"
    beta = cast(zeros(1,n+1), precision);
    for j = 0:n
        beta(j+1) = (-1)^j*exp(gamma(n+1)-gamma(j+1)-gamma(n-j+1));
    end

elseif mesh_type == "chebychev1"
    beta = ((-1).^k) .* sin((2*k + 1) * pi / (2*n + 2));
```

```
elseif mesh_type == "chebychev2"
    beta = cast(ones(1, n+1), precision);
    beta(1) = 0.5;
    beta(end) = 0.5;
    beta = beta .* ((-1).^k);
end

beta = cast(beta, precision);
end
```

4.4 Barycentric-2 Polynomial Evaluation

The routine for evaluation the polynomial $p_n(x)$ implements that formula from Equation (2). For each x that gets evaluated, the routine computes $n+1$ terms from $\rho_i = \frac{\beta_i}{x-x_i}$ and then preforms the summations required for the numerator $\sum \rho_i f_i$ and the denominator $\sum \rho_i$. Each summation then involves $n+1$ terms bringing a total complexity of $O(n)$ floating point operations per evaluation point.

Similar to the Barycentric-1 case, a conditional check is needed to ensure $fl(x - x_i) = 0$ does not happen. If $x = x_i$ then the routine returns the function evaluation f_i to handle the potential singularity that the nodes can cause. While the two barycentric forms are mathematically equivalent, they produce differing numerical properties as seen later in the results section of the report.

```
function px = Bary2_Interp(evaluation_points,x_mesh,beta,fx,prec)

px = cast(zeros(size(evaluation_points)), precision);

evaluation_points = cast(evaluation_points(:), precision);
x_mesh = cast(x_mesh(:)', precision);
beta = cast(beta(:)', precision);
fx = cast(fx(:)', precision);

for j=1:length(evaluation_points)
    x_current = evaluation_points(j);

    difference = x_current - x_mesh;
    exact_spot = find(difference == 0,1);

    if ~isempty(exact_spot)
        px(j) = fx(exact_spot);
    else
        rho = beta ./ difference;
        px(j) = sum(rho .* fx)/ sum(rho);
    end
```

```
end
end
```

4.5 Newton Form Coefficient Generation

The implementation of the Newton form involves generating the basis coefficients α_i and evaluating the function f_i. The routine takes the then computed $f(x_i)$ and iteratively updates them for the divided difference calculations using a nested loop where the outer loop iterates through j, while the inner loop updates the coefficients downward from n to j. This leads to time complexity of $O(n^2)$ due to the nested loops. But, by iterating backwards the same vector is overwritten which enables the routine to achieve $O(n)$ space complexity.

```
function [fx, coefficients] = NewtonCoeff(mesh,f,precision)
    n = length(mesh);
    fx = cast(f(mesh), precision);
    coeff = double(fx);

    for j = 2:n
        for i = n:-1:j
            coeff(i)=(coeff(i)-coeff(i-1))/(mesh(i)-mesh(i-j+1));
        end
    end
    coeff = cast(coeff, precision);
end
```

Another way of possibly computing and storing the coefficients is through the divided difference table. In this method the table is a 2-D matrix that stores all the possible coefficients that can be used to evaluate the polynomial. The downside to this method is the increase in space complexity $O(n^2)$ compared to the first method. The table is for useful as it shows intermediate values for every possible path and can be more beneficial to debugging, but in a production setting, the first routine for evaluating the Newton Coefficients is preferred for reduced complexity.

4.6 Newton Form Polynomial Evaluation

The evaluation of the polynomial for Newtons form utilizes the adapted Horner's rule. The routine uses a nested loop to evaluate:

$$p_n(x) = \alpha_0 + \alpha_1(x - x_0) + \alpha_2(x - x_0)(x - x_1) + ... + \alpha_n(x - x_0)...(x - x_{n-1}) \quad (9)$$

Starting from the highest order coefficient α_n the iteration works backwards: $s = s(x - x_i) + \alpha_i$. Therefore the total complexity of this method is $O(n)$ floating point operations. The routine is also able to remain versatile for mesh points

that are all set to the same value. In this case the polynomial is effectively evaluated using the monomial basis $\{x^k\}_{k=o}^{n}$

Newton's Form is also highly dependent upon the ordering of the mesh when run in single point precision as the method is sensitive to the applied sequences: Increasing, Decreasing, and Leja's Ordering. Like mentioned earlier, the round-off that occurs will cause the trajectories of the method to vary. This is why Leja's Ordering is helpful for maintain the growth of the basis function leading to an improvement in stability.

```
function px = Newton_Inter(eval_pts,mesh,coeff,prec)

evaluation_points = cast(evaluation_points, precision);
mesh = cast(mesh, precision);
coeff  = cast(coeff, precision);

n = length(coeff) - 1;
px = cast(zeros(size(evaluation_points)), precision);

for j = 1:length(evaluation_points)
    x = evaluation_points(j);
    s = coeff(n + 1);

    for i = n:-1:1
        s = s * (x - mesh(i)) + coeff(i);
    end

    px(j) = s;
end
end
```

The algorithm for computing the ordering uses the built-in sorting function to achieve the first two orderings. Leja's ordering distributes the nodes such that the basis function does not produce unwanted numerical overflow. The Algorithm begins by initialize the first point as the point with the highest absolute value: $x_0 = argmax|x_i|$. Then for each subsequent position k, the remaining mesh point are searched and the one that maximizes the product of distances to the previous point in the new sequence: $argmax \prod_{j=1}^{k-1} |x - x_j|$. The selected point is removed from the pool of options and the process repeats until each point from the old sequence is ordered in the new sequence. The nested structure yields $O(n^2)$ time complexity and $O(n)$ space complexity due to all operations taking place on the provided vector.

```
function ordered_mesh = Ordering(mesh, type)
 n = length(mesh);
if type == "increasing"
```

```
    ordered_mesh = sort(mesh, "ascend");
elseif type == "decreasing"
    ordered_mesh = sort(mesh, "descend");
elseif type == "leja"
    ordered_mesh = zeros(size(mesh));
    remaining = mesh;

    [~, idx] = max(abs(remaining));
    ordered_mesh(1) = remaining(idx);
    remaining(idx) = [];

    for k = 2:n
        [~, idx] = max(prod(abs(remaining' - ordered_mesh(1:k-1)), 2));
        ordered_mesh(k) = remaining(idx);
        remaining(idx) = [];
    end
else
    ordered_mesh = mesh;
end
end
```

4.7 Conditioning Analysis

The developed conditioning routine is used to quantify the sensitivity of the interpolation problem to slight perturbations within the data. The routine computes the Absolute Condition Number $k_1 = \kappa(x, n, 1)$ where the maximum value of the function over the desired interval is the Lebesgue Constant Λ_n. Additionally, the Relative Condition Number $\kappa_2 = \kappa(x, n, y)$ is computed, assessing the local sensitivity of the function values magnitude.

To ensure accuracy, the routine begins by forcing all calculations into double precision, regardless of the parameter setting. The routine then computes the Barycentric-1 weights γ_i. The weights are then used to represent the Lagrange Basis function $l_i(x)$. Then for each point being evaluated the routine looks for singularities, computes the product of the nodes, and computes the basis function. Then to find κ_1, κ_2 take the sum of the absolute values

$$l(x) = \prod(x - x_i), \quad l_i(x) = l(x)\frac{\gamma_i}{x - x_i} \tag{10}$$

$$\kappa_1 = |l(x)| \sum |\frac{\gamma_i}{x - x_i}|, \quad \kappa_2 = \frac{|l(x) \sum \frac{\gamma_i f_i}{x - x_i}|}{|p_n(x)|} \tag{11}$$

The computational complexity of the weight stays $O(n^2)$ like the Barycentric-1 Coefficient Generation. The evaluation of the conditioning number has a total complexity of $O(mn)$ for each of the m evaluation points and n operations for

computes the summation. The overall space complexity is $O(m+n)$ used to store the mesh, weights, and condition numbers.

This routine is a strong analytical tool that allows for verification of Λ_n and H_n. Comparing across the different meshes can help explain why interpolation error blows up on a uniform grid but remains stable on Chebyshev points.

```
function [px,kappa1,kappa2]=Bary_Condition(eval_pts,mesh,fx)
xx = double(evaluation_points(:));
x_i = double(mesh(:));
f_i = double(fx(:));
n = length(x_i)-1;

gamma = ones(1, n+1);

for j = 1:n+1
    for k = 1:n+1
        if j ~= k
            gamma(j) = gamma(j) * (x_i(j)-x_i(k));
        end
    end
end
gamma = 1 ./ gamma;
gamma = gamma(:);

px = zeros(size(xx));
kappa1 = zeros(size(xx));
kappa2 = zeros(size(xx));

for j = 1:length(xx)
    x = xx(j);
    difference = x - x_i;

    exact_spot = find(difference == 0, 1);
    if ~isempty(exact_spot)
        px(j) = f_i(exact_spot);
        kappa1(j) = 1.0;
        kappa2(j) = 1.0;
    else
        lx = prod(difference);
        coefficients = gamma ./ difference;

        px(j) = lx * sum(coefficients .* f_i);

        kappa1(j) = abs(lx) * sum(abs(coefficients));
        numerator = abs(lx) * sum(abs(coefficients .* f_i));
```

```
        kappa2(j) = numerator / abs(px(j));
    end
end
end
```

5 Experimental Design and Results

The following section describes the systematic design and evaluation of polynomial interpolation across 3 distinct interpolating forms (Barycentric-1, Barycentric-2, and Newton Form), three different mesh distributions (uniform, Chebyshev of the 1st kind, and Chebyshev of the 2nd kind), along three different node orderings for Newton's Form (increasing, decreasing, and Leja). All experiments related to stability and accuracy are computed with single precision, while double precision is the exact reference value used in the error analysis.

The four main metrics reported through the experiments are:

- Infinity Norm Error: $||r(x)||_\infty = ||f(x) - p_n(x)||_\infty = max_x|f(x) - p_n(x)|$
- Relative Infinity Norm: $||r(x)||_\infty^{relative} = \frac{||f(x)-p_n(x)||_\infty}{||f(x)||_\infty}$
- Lebesgue Constant: $\Lambda_n = max_x\kappa(x, n, 1) = max_x \sum |l_i(x)|$
- Conditioning Number: $H_n = max_x\kappa(x, n, y)$

The infinity norm error measures the worst case deviation from the computed interpolating form to the exact function over the evaluation points. The relative infinity norm normalizes and scales according to the function, allowing helpful insights and comparisons across different magnitudes. The Lebesgue constant measures the worst-case amplification of perturbations in the function values y_i. The conditioning number measures the sensitivity to the specific interpolation problem. Together, the Lebesgue constant and the condition number bound the error on single point precision.

$$||f(x) - p_n(x)||_\infty \leq H_n\mu + ||f - p_n||_\infty, \quad \mu = \text{Unit Round-off} \tag{12}$$

Providing a strong theoretical benchmark to assess the errors against.

5.1 Task 2: Testing Results for $f_1(x) = (x - 2)^9$

The main objective of this task to evaluate the numerical behavior that different interpolating forms and meshes for a 9th degree polynomial on the interval [-1,1].

Conditioning Analysis: Looking at the data collected in table 1. A uniform mesh produced a Lebesgue constant of approximately 1.76×10^1 and a conditioning number around 7.63×10^3. While Λ_n is somewhat low, the higher

value from H_n suggests that the uniform distribution is more sensitive than one with clusters. Looking next at both of the Chebyshev points, they produce much lower values for Λ_n and H_n. From this we can infer that the clustered meshes are much less sensitive to perturbation in the data. This makes sense for the Chebyshev points due to the fact that they are clustered the the endpoints of the interval, minimizing the affects from the Runge phenomenon. This is why the environment is more stable when Chebyshev points are utilized.

Single Precision Stability and Accuracy: Using the relative infinity norm as the measure for accuracy between single and double point precision. For the Barycentric-2 form across each mesh the interpolation method appears to be consistently stable matching the theory associated with this form. Looking next at Newton's form, the increasing and decreasing mesh have similar error across the board. Comparing this to Leja's ordering there is an consistent bias that it is more stable as seen in the lower error values. This is especially seen for the case on the uniform mesh. In addition, the data produced is also supported by the error in single precision being bounded by $H_n\mu$. We can infer from this that the sensitivity to the ordering of the nodes is due to the rounding errors being propagated through the computations for the divided difference. Leja's ordering prevents the basis function from producing overflow from rapid growth.

5.2 Task 3: Testing Results for $f_2(x;d) = \prod_{i=1}^{d}(x-i)$

This tasks main objective is to investigate f_2 on the interval $[0, d+1]$ for the polynomial of degree d.

Conditioning Analysis: Analyzing the Lebesgue Constant from table 2, as the degree d increases from $5 \to 29$ on a uniform mesh the value of the constant increases at a significant pace. Additionally, the condition number starts off extremely high and does not decrease as the degree of the polynomial increases. From this we are able to tell that this problem is extremely sensitive to perturbation from the function value. We can make sense of this from the fact that the distance between the roots of f_2 increases as the degree d increases. This means that regardless how hard we try to maintain the accuracy of the interpolation, the problem is ill-conditioned so all methods tested are going to struggle.

Single Precision Stability and Accuracy: The Barycentric-2 form yielded consistent increases in the infinity norm and relative infinity norm as the degree d of the polynomial grew. For Newton's Form with the increasing and decreasing ordering that the polynomial fails with exceedingly high error values, regardless of the mesh type. For Leja's ordering there is a significant drop off in the order of the magnitudes allowing for a much more stable relative error value. The failure of the standard orderings happens from the rapid growth of the basis function. In single point precision this leads to numerical overflow. Leja's ordering is able to defend against this by maximizing the distance between the nodes, helping with the overflow problem.

5.3 Task 4: Testing Results for $f_3(x) = \ell_n(x)$

This tasks objective is to evaluate and report on the behavior associated with the Lagrange basis function that satisfies:

$$\ell_n(x_i) = \begin{cases} 1 & \text{if } i = n \\ 0 & \text{Otherwise} \end{cases} \tag{13}$$

Conditioning Analysis: First observing Λ_n on a uniform mesh in table 3, there is constant growth in the order of the magnitude as the degree n goes from $20 \to 29$. This is strikingly different from the results collected from the Chebyshev points which exhibit practically no change as the degree increases from the already low constant. The conditioning number H_n is constantly at 1 for each test. This is to be expected as the function produces 0's or 1's. The sensitivity in this problem stems from the nodes distribution. This explains why Λ_n grows on the uniform mesh due to the Runge phenomenon associated with the Lagrange basis.

Single Precision Stability and Accuracy: Continuing to analyze the results from table 3, the accuracy is assessed via the relative infinity norm of the residual $||r(x)||_\infty^{relative}$. Using the Chebyshev nodes, the Barycentric-2 Form produces relative error in accordance with single point precision machine epsilon, but struggle to keep that low error on a uniform mesh. For Newton's Form on the uniform mesh, the increasing sequence was the best performer while Leja's ordering struggled significantly more. Then when the mesh points are of the Chebyshev form, the decreasing ordering shows the lowest residual error. This odd behavior seems to be related to the maximum height of the basis function. For Newton's form, if the nodes near that peak value are processed in such a way that the divided difference does not grow fast enough then that stability remains in tact. While the function f_3 is extremely stable and conditioned well, the choice of mesh is still the dominant factor in determining the stability of the interpolation process.

5.4 Task 5: Testing Results for $f_4(x) = \frac{1}{1+25x^2}$

The final task is the interpolation of the Runge function on the interval $[-1, 1]$. This function is purposely designed to showcase the Runge phenomenon. Additionally for this task convergence properties are investigated on clustered meshes.

Conditioning Analysis: On the uniform mesh, the Lebesgue constant grows in several order of magnitude as n increases. Thus the problem showcases signs of ill-conditioning on the uniform mesh. On the other hand, the Chebyshev mesh does not increase the Lebesgue constant by any degrees in magnitude. Similarly for the condition number H_n, the uniform mesh produces a higher conditioning number compared to the Chebyshev points. This illustrates why the uniform mesh does not do well interpolating the Runge function.

From the results of Lebesgue constant one can infer that the small errors from rounding are amplified which leads to the large residual values.

Single Precision Stability and Accuracy: Newton's form on the uniform mesh with the increasing and decreasing ordering produce node error values around 2.13×10^2 while Leja's ordering kept the node errors lower at around 5.70×10^{-7}. Looking at the Chebyshev nodes next, Newton's form fails to reach low relative error values. Comparatively, when evaluated with Leja's ordering, Newton's form produces an error on par of that with Barycentric-2. This demonstrate that even when the optimal mesh is used, the ordering can still drastically affect the final results.

Convergence Analysis: The program also asks that convergence is analyzed from $n = 5, ..., 100$ using the results from Table 5.

- Uniform Points: As seen through the table, the error grows quickly with the results reaching ∞ by the time $n = 50$. Once $n = 60$ the results are no longer available and NaN's are produced. This validate theory that Runge's function $p_n(x)$ does not converge to $f(x)$ for the uniform mesh.
- Chebyshev Points: Both of the Chebyshev points demonstrate the consistent convergence. This is interpreted to mean that in order to achieve an approximation of 10^{-6} the the degree of the polynomial must be $n \approx 60$ and so on.
- Precision Threshold: When $n = 100$ for the Chebyshev points the error hits around 2.23×10^{-7}. Therefore looking at the table values, once the interpolation values reach this point there is a diminishing return as more noise is added without significant increase. Additionally this value is approaching unit round-off values for single precision arithmetic and cannot go below this value.

This experiment confirms that if the poles of a function are near the interval of interpolation then the uniform mesh will diverge. In stark contrast, The Chebyshev points paired with the numerical stability of the Barycentric-2 form or Leja ordered Newton form can produce results with extremely high accuracy.

6 Conclusion

This report evaluated the numerical performance and stability of Barycentric-1, Barycentric-2, and Newton's interpolating form across distinct mesh points and orderings. The Algorithms were tested across 4 different functions that each showcase a unique aspect of the interpolation. These functions helped to generate the key insights produced in this report about conditioning and numerical stability. The experiments consistently showcased that the node distribution was the most important factor in assessing the accuracy of the method. Single

point precision was also used as a way to test the limitations of each method. Summarizing everything together, in order to evaluate a high degree polynomial reliably one should have: An optimal mesh distribution, a stable mathematical derivation, and a node ordering that works in your favor.

Table 1: Empirical Results for $f_1(x) = (x-2)^9$, $n = 9$, Interval $[-1, 1]$

Mesh	**Method**	**Order**	$\boldsymbol{\Lambda_n}$	$\boldsymbol{H_n}$	$\|\boldsymbol{r}\|_\infty$	**Rel.** $\|\boldsymbol{r}\|_\infty$	**RMS Error**	**Node Error**
Uniform	Newton	Increasing	1.76×10^1	7.63×10^3	1.19×10^{-2}	6.06×10^{-7}	4.19×10^{-3}	1.14×10^{-2}
Uniform	Newton	Decreasing	1.76×10^1	7.63×10^3	1.20×10^{-2}	6.11×10^{-7}	2.72×10^{-3}	1.95×10^{-3}
Uniform	Newton	Leja	1.76×10^1	7.63×10^3	6.79×10^{-3}	3.45×10^{-7}	2.13×10^{-3}	1.46×10^{-3}
Uniform	Bary-2	—	1.76×10^1	7.63×10^3	1.25×10^{-2}	6.33×10^{-7}	2.70×10^{-3}	—
Cheb-1	Newton	Increasing	2.43×10^0	1.14×10^3	2.28×10^{-2}	1.16×10^{-6}	5.90×10^{-3}	1.18×10^{-2}
Cheb-1	Newton	Decreasing	2.43×10^0	1.14×10^3	4.57×10^{-3}	2.32×10^{-7}	1.04×10^{-3}	1.95×10^{-3}
Cheb-1	Newton	Leja	2.43×10^0	1.14×10^3	6.31×10^{-3}	3.21×10^{-7}	1.99×10^{-3}	1.95×10^{-3}
Cheb-1	Bary-2	—	2.43×10^0	1.14×10^3	4.57×10^{-3}	$\mathbf{2.32 \times 10^{-7}}$	1.13×10^{-3}	—
Cheb-2	Newton	Increasing	2.36×10^0	4.48×10^2	2.29×10^{-2}	1.16×10^{-6}	5.31×10^{-3}	1.23×10^{-2}
Cheb-2	Newton	Decreasing	2.36×10^0	4.48×10^2	9.10×10^{-3}	4.62×10^{-7}	1.85×10^{-3}	9.77×10^{-4}
Cheb-2	Newton	Leja	2.36×10^0	4.48×10^2	1.10×10^{-2}	5.61×10^{-7}	3.60×10^{-3}	3.91×10^{-3}
Cheb-2	Bary-2	—	2.36×10^0	4.48×10^2	8.74×10^{-3}	4.44×10^{-7}	2.08×10^{-3}	—

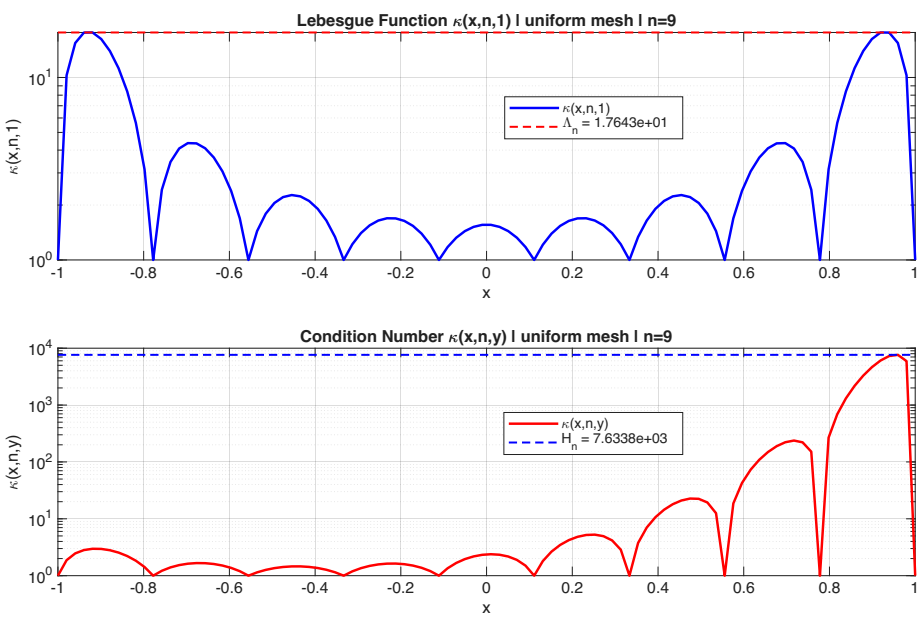

Figure 1: Task 2: Conditioning Plot

Table 2: Empirical Results for $f_2(x;d) = \prod_{i=1}^{d}(x-i)$, Interval $[0, d+1]$

Mesh	**Method**	**Order**	$\boldsymbol{d}$	$\boldsymbol{\Lambda_n}$	$\boldsymbol{H_n}$	$\|\boldsymbol{r}\|_\infty$	**Rel.** $\|\boldsymbol{r}\|_\infty$
Uniform	Newton	Increasing	5	3.10×10^{0}	3.82×10^{16}	5.08×10^{-5}	4.24×10^{-7}
Uniform	Newton	Decreasing	5	3.10×10^{0}	1.91×10^{16}	7.37×10^{-5}	6.15×10^{-7}
Uniform	Newton	Leja	5	3.10×10^{0}	2.18×10^{16}	4.03×10^{-5}	3.36×10^{-7}
Uniform	Bary-2	—	5	3.10×10^{0}	3.82×10^{16}	4.04×10^{-5}	3.37×10^{-7}
Uniform	Newton	Increasing	20	1.07×10^{4}	4.43×10^{15}	1.27×10^{16}	5.22×10^{-3}
Uniform	Newton	Decreasing	20	1.07×10^{4}	1.33×10^{16}	3.39×10^{16}	1.39×10^{-2}
Uniform	Newton	Leja	20	1.07×10^{4}	1.22×10^{16}	5.86×10^{12}	$\mathbf{2.41 \times 10^{-6}}$
Uniform	Bary-2	—	20	1.07×10^{4}	4.43×10^{15}	1.26×10^{14}	5.16×10^{-5}
Uniform	Newton	Increasing	29	3.33×10^{6}	1.30×10^{16}	2.70×10^{31}	3.05×10^{0}
Uniform	Newton	Decreasing	29	3.33×10^{6}	7.80×10^{15}	2.84×10^{31}	3.21×10^{0}
Uniform	Newton	Leja	29	3.33×10^{6}	1.78×10^{16}	1.73×10^{27}	$\mathbf{1.96 \times 10^{-4}}$
Uniform	Bary-2	—	29	3.33×10^{6}	1.30×10^{16}	5.66×10^{28}	6.40×10^{-3}
Cheb-1	Newton	Increasing	20	2.90×10^{0}	1.85×10^{16}	8.58×10^{17}	3.53×10^{-1}
Cheb-1	Newton	Decreasing	20	2.90×10^{0}	3.70×10^{16}	5.79×10^{17}	2.38×10^{-1}
Cheb-1	Newton	Leja	20	2.90×10^{0}	4.69×10^{15}	2.91×10^{12}	$\mathbf{1.20 \times 10^{-6}}$
Cheb-1	Bary-2	—	20	2.90×10^{0}	1.85×10^{16}	3.53×10^{12}	$\mathbf{1.45 \times 10^{-6}}$
Cheb-1	Newton	Increasing	29	3.13×10^{0}	1.96×10^{16}	1.15×10^{34}	1.31×10^{3}
Cheb-1	Newton	Decreasing	29	3.13×10^{0}	4.89×10^{15}	1.07×10^{34}	1.21×10^{3}
Cheb-1	Newton	Leja	29	3.13×10^{0}	3.12×10^{15}	6.39×10^{24}	$\mathbf{7.22 \times 10^{-7}}$
Cheb-1	Bary-2	—	29	3.13×10^{0}	1.96×10^{16}	4.96×10^{24}	$\mathbf{5.61 \times 10^{-7}}$
Cheb-2	Newton	Increasing	20	2.85×10^{0}	9.12×10^{15}	2.15×10^{18}	8.82×10^{-1}
Cheb-2	Newton	Decreasing	20	2.85×10^{0}	∞	1.45×10^{18}	5.95×10^{-1}
Cheb-2	Newton	Leja	20	2.85×10^{0}	1.82×10^{16}	4.08×10^{12}	$\mathbf{1.68 \times 10^{-6}}$
Cheb-2	Bary-2	—	20	2.85×10^{0}	9.12×10^{15}	3.46×10^{12}	$\mathbf{1.42 \times 10^{-6}}$
Cheb-2	Newton	Increasing	29	3.09×10^{0}	2.46×10^{16}	1.48×10^{34}	1.67×10^{3}
Cheb-2	Newton	Decreasing	29	3.09×10^{0}	1.64×10^{16}	9.14×10^{33}	1.03×10^{3}
Cheb-2	Newton	Leja	29	3.09×10^{0}	2.46×10^{16}	3.29×10^{24}	$\mathbf{3.72 \times 10^{-7}}$
Cheb-2	Bary-2	—	29	3.09×10^{0}	2.46×10^{16}	2.71×10^{24}	$\mathbf{3.07 \times 10^{-7}}$

Table 3: Empirical Results for $f_3(x) = \ell_n(x)$ (Lagrange Basis), Interval $[-1, 1]$

Mesh	**Method**	**Order**	n	Λ_n	H_n	$\|r\|_\infty$	**Rel.** $\|r\|_\infty$	**Node Error**
Uniform	Bary-2	—	20	1.07×10^4	1.00	6.74×10^{-6}	6.74×10^{-6}	—
Uniform	Newton	Increasing	20	1.07×10^4	1.00	3.35×10^{-7}	$\mathbf{3.35 \times 10^{-7}}$	2.38×10^{-7}
Uniform	Newton	Decreasing	20	1.07×10^4	1.00	1.00×10^0	1.00×10^0	2.38×10^{-7}
Uniform	Newton	Leja	20	1.07×10^4	1.00	1.80×10^3	1.80×10^3	2.38×10^{-7}
Uniform	Bary-2	—	29	3.33×10^6	1.00	2.12×10^{-2}	2.12×10^{-2}	—
Uniform	Newton	Increasing	29	3.33×10^6	1.00	4.16×10^{-7}	$\mathbf{4.16 \times 10^{-7}}$	2.98×10^{-7}
Uniform	Newton	Decreasing	29	3.33×10^6	1.00	1.00×10^0	1.00×10^0	2.98×10^{-7}
Uniform	Newton	Leja	29	3.33×10^6	1.00	4.52×10^5	4.52×10^5	1.19×10^{-7}
Cheb-1	Bary-2	—	20	2.90×10^0	1.00	3.82×10^{-6}	3.00×10^{-6}	—
Cheb-1	Newton	Increasing	20	2.90×10^0	1.00	1.27×10^0	9.99×10^{-1}	1.19×10^{-7}
Cheb-1	Newton	Decreasing	20	2.90×10^0	1.00	3.47×10^{-6}	$\mathbf{2.72 \times 10^{-6}}$	1.19×10^{-7}
Cheb-1	Newton	Leja	20	2.90×10^0	1.00	1.33×10^0	1.04×10^0	3.58×10^{-7}
Cheb-1	Bary-2	—	29	3.13×10^0	1.00	7.13×10^{-6}	5.60×10^{-6}	—
Cheb-1	Newton	Increasing	29	3.13×10^0	1.00	1.27×10^0	1.00×10^0	5.96×10^{-8}
Cheb-1	Newton	Decreasing	29	3.13×10^0	1.00	6.06×10^{-6}	$\mathbf{4.76 \times 10^{-6}}$	5.96×10^{-8}
Cheb-1	Newton	Leja	29	3.13×10^0	1.00	1.24×10^0	9.72×10^{-1}	1.79×10^{-7}
Cheb-2	Bary-2	—	20	2.85×10^0	1.00	7.00×10^{-7}	$\mathbf{7.00 \times 10^{-7}}$	—
Cheb-2	Newton	Increasing	20	2.85×10^0	1.00	1.00×10^0	1.00×10^0	0.00×10^0
Cheb-2	Newton	Decreasing	20	2.85×10^0	1.00	1.10×10^{-6}	$\mathbf{1.10 \times 10^{-6}}$	0.00×10^0
Cheb-2	Newton	Leja	20	2.85×10^0	1.00	1.00×10^0	1.00×10^0	0.00×10^0
Cheb-2	Bary-2	—	29	3.09×10^0	1.00	4.90×10^{-7}	$\mathbf{4.90 \times 10^{-7}}$	—
Cheb-2	Newton	Increasing	29	3.09×10^0	1.00	1.00×10^0	1.00×10^0	5.96×10^{-8}
Cheb-2	Newton	Decreasing	29	3.09×10^0	1.00	7.43×10^{-7}	$\mathbf{7.43 \times 10^{-7}}$	5.96×10^{-8}
Cheb-2	Newton	Leja	29	3.09×10^0	1.00	1.00×10^0	1.00×10^0	0.00×10^0

Table 4: Empirical Results for $f_4(x) = \frac{1}{1+25x^2}$ (Runge Function), Interval $[-1, 1]$

Mesh	**Method**	**Order**	n	Λ_n	H_n	$\|r\|_\infty$	**Rel.** $\|r\|_\infty$	**RMS Error**	**Node Error**
Uniform	Bary-2	—	10	2.99×10^1	4.24×10^1	1.92×10^0	1.92×10^0	5.77×10^{-1}	—
Uniform	Newton	Increasing	10	2.99×10^1	4.24×10^1	1.92×10^0	1.92×10^0	5.77×10^{-1}	2.74×10^{-6}
Uniform	Newton	Decreasing	10	2.99×10^1	4.24×10^1	1.92×10^0	1.92×10^0	5.77×10^{-1}	2.74×10^{-6}
Uniform	Newton	Leja	10	2.99×10^1	4.24×10^1	1.92×10^0	1.92×10^0	5.77×10^{-1}	1.34×10^{-7}
Uniform	Bary-2	—	30	6.28×10^6	9.32×10^3	2.22×10^3	2.22×10^3	3.39×10^2	—
Uniform	Newton	Increasing	30	6.28×10^6	9.32×10^3	2.41×10^3	2.42×10^3	3.56×10^2	2.13×10^2
Uniform	Newton	Decreasing	30	6.28×10^6	9.32×10^3	2.41×10^3	2.42×10^3	3.56×10^2	2.13×10^2
Uniform	Newton	Leja	30	6.28×10^6	9.32×10^3	2.27×10^3	2.28×10^3	3.49×10^2	5.70×10^{-7}
Cheb-1	Bary-2	—	10	2.49×10^0	1.19×10^2	1.09×10^{-1}	1.09×10^{-1}	5.68×10^{-2}	—
Cheb-1	Newton	Increasing	10	2.49×10^0	1.19×10^2	1.09×10^{-1}	1.09×10^{-1}	5.68×10^{-2}	1.25×10^{-5}
Cheb-1	Newton	Decreasing	10	2.49×10^0	1.19×10^2	1.09×10^{-1}	1.09×10^{-1}	5.68×10^{-2}	1.25×10^{-5}
Cheb-1	Newton	Leja	10	2.49×10^0	1.19×10^2	1.09×10^{-1}	1.09×10^{-1}	5.68×10^{-2}	2.20×10^{-7}
Cheb-1	Bary-2	—	30	3.15×10^0	7.68×10^0	2.05×10^{-3}	$\mathbf{2.06 \times 10^{-3}}$	1.03×10^{-3}	—
Cheb-1	Newton	Increasing	30	3.15×10^0	7.68×10^0	1.24×10^1	1.24×10^1	1.47×10^0	5.68×10^0
Cheb-1	Newton	Decreasing	30	3.15×10^0	7.68×10^0	1.24×10^1	1.24×10^1	1.47×10^0	5.68×10^0
Cheb-1	Newton	Leja	30	3.15×10^0	7.68×10^0	2.05×10^{-3}	$\mathbf{2.06 \times 10^{-3}}$	1.03×10^{-3}	8.90×10^{-7}

Table 5: Convergence of $\|f_4 - p_n\|_\infty$ for Barycentric Form 2

n	**Uniform**	**Cheb-1**	**Cheb-2**
5	4.32×10^{-1}	5.56×10^{-1}	6.38×10^{-1}
10	1.92×10^{0}	1.09×10^{-1}	1.32×10^{-1}
15	2.10×10^{0}	8.30×10^{-2}	9.92×10^{-2}
20	5.97×10^{1}	1.53×10^{-2}	1.77×10^{-2}
25	7.61×10^{1}	1.14×10^{-2}	1.36×10^{-2}
30	2.24×10^{3}	2.06×10^{-3}	2.43×10^{-3}
40	2.45×10^{5}	2.90×10^{-4}	3.39×10^{-4}
50	∞	3.92×10^{-5}	4.62×10^{-5}
60	NaN	5.39×10^{-6}	6.33×10^{-6}
80	NaN	3.68×10^{-7}	4.81×10^{-7}
100	NaN	2.23×10^{-7}	2.73×10^{-7}

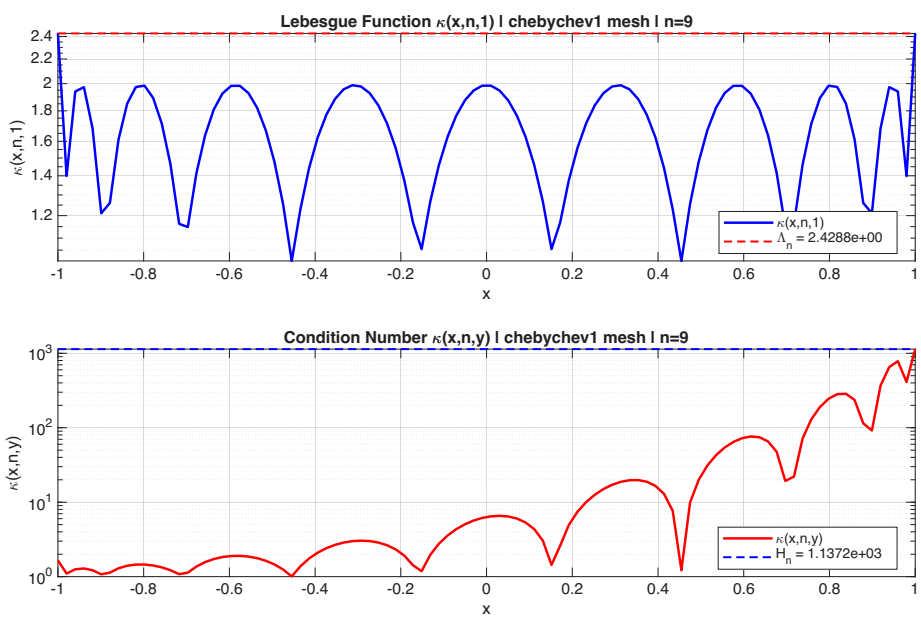

Figure 2: Task 2: Conditioning Plot

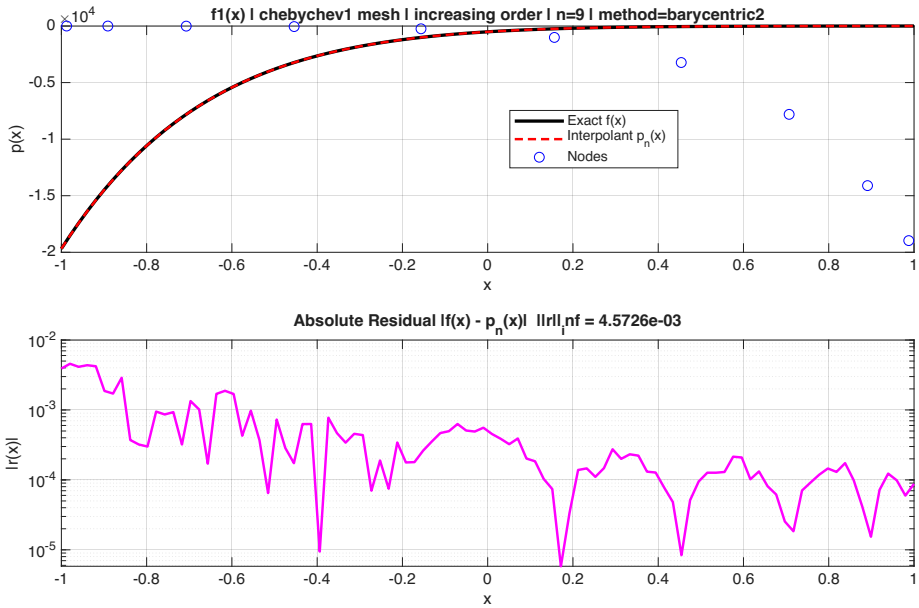

Figure 3: Task 2: Interpolation and Residual Plot

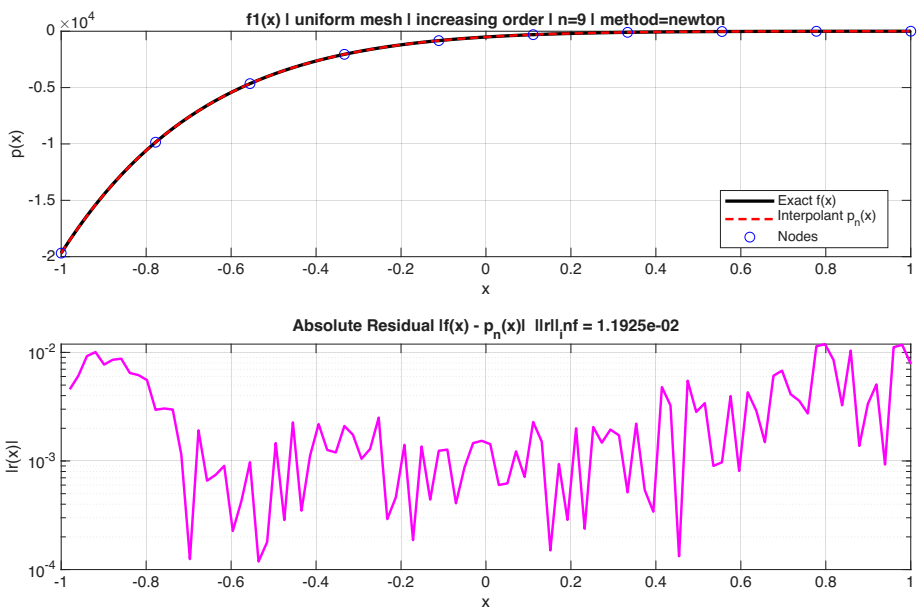

Figure 4: Task 2: Interpolation and Residual Plot

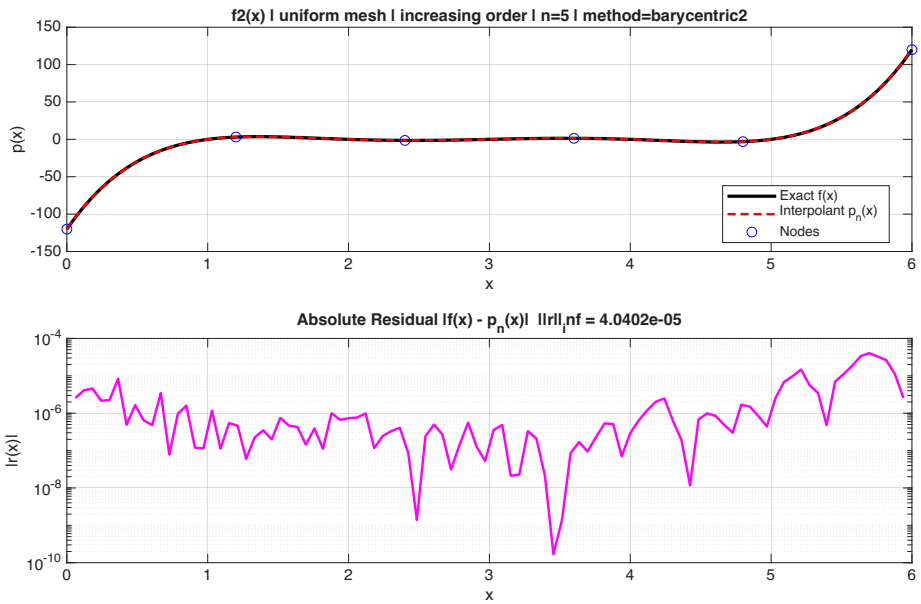

Figure 5: Task 3: Interpolation and Residual Plot

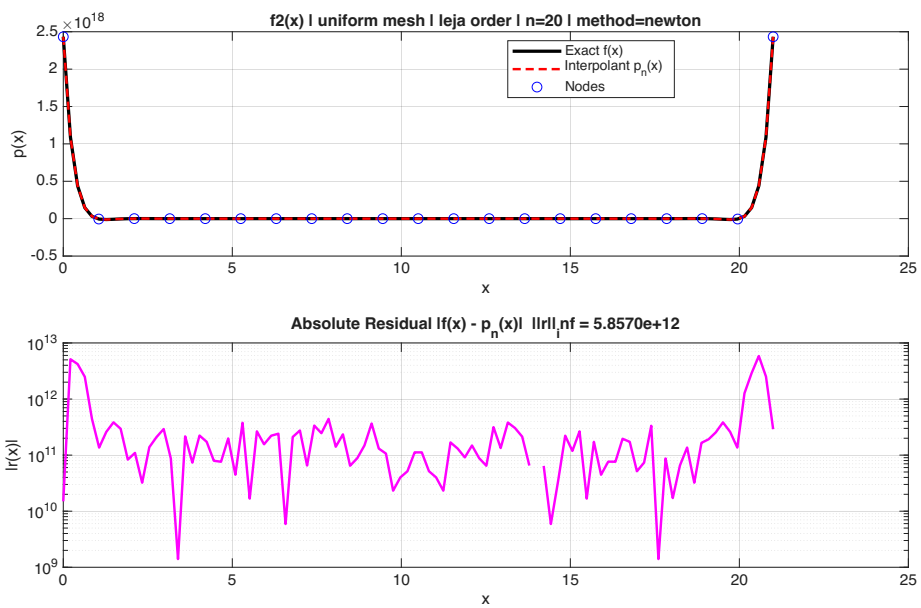

Figure 6: Task 3: Interpolation and Residual Plot

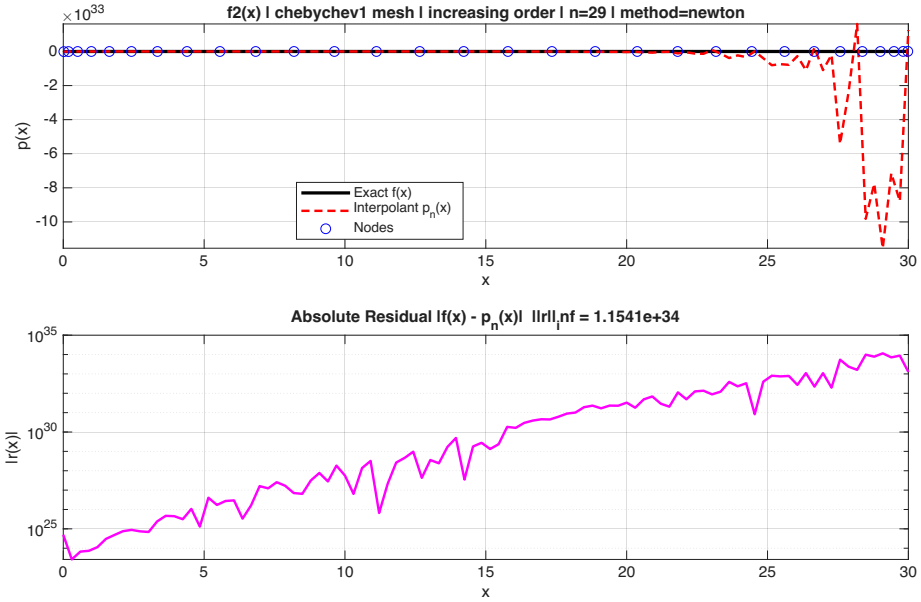

Figure 7: Task 3: Interpolation and Residual Plot

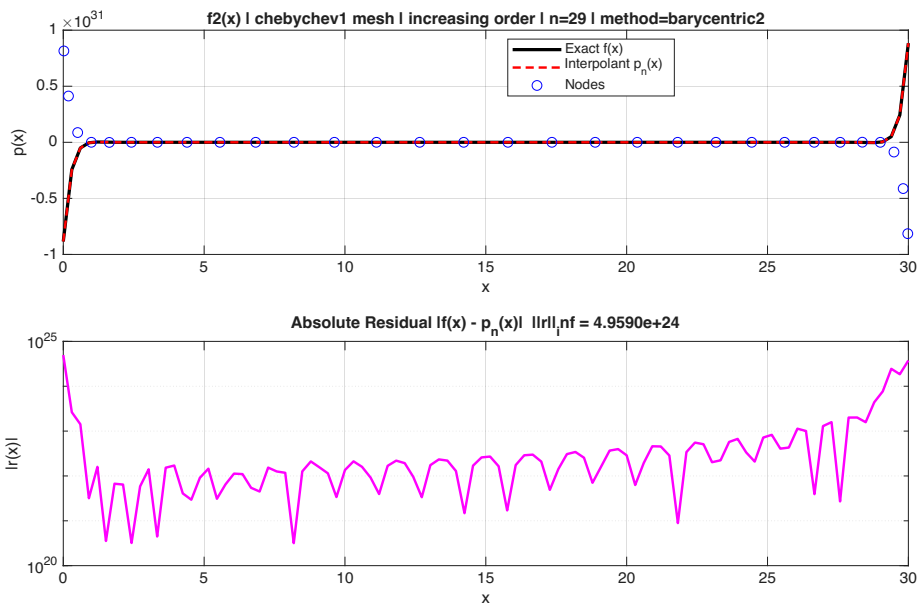

Figure 8: Task 3: Interpolation and Residual Plot

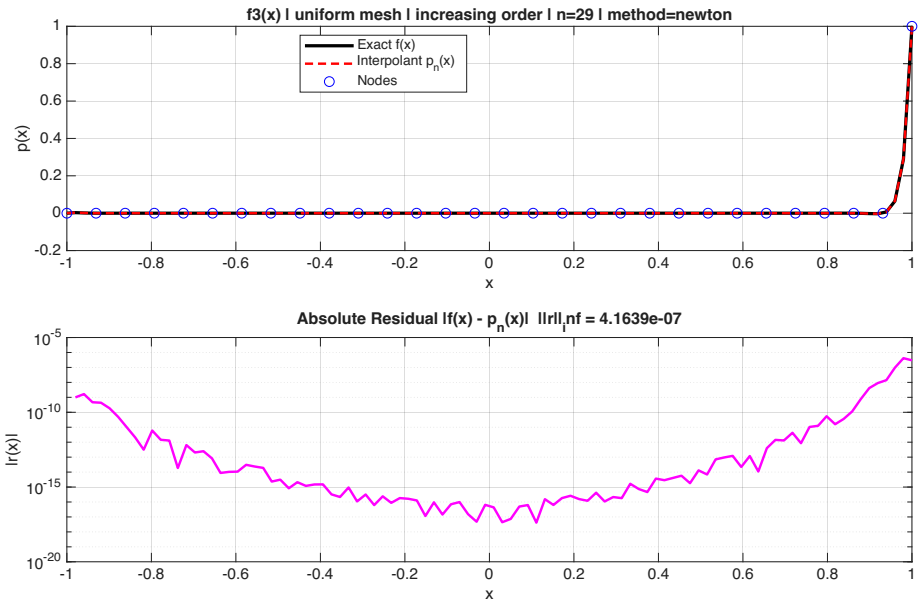

Figure 9: Task 4: Interpolation and Residual Plot

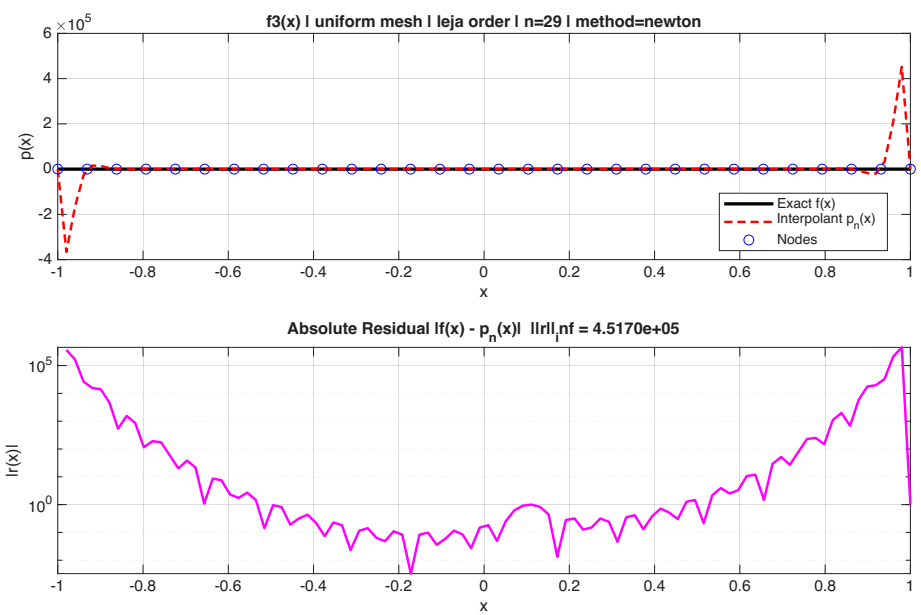

Figure 10: Task 4: Interpolation and Residual Plot

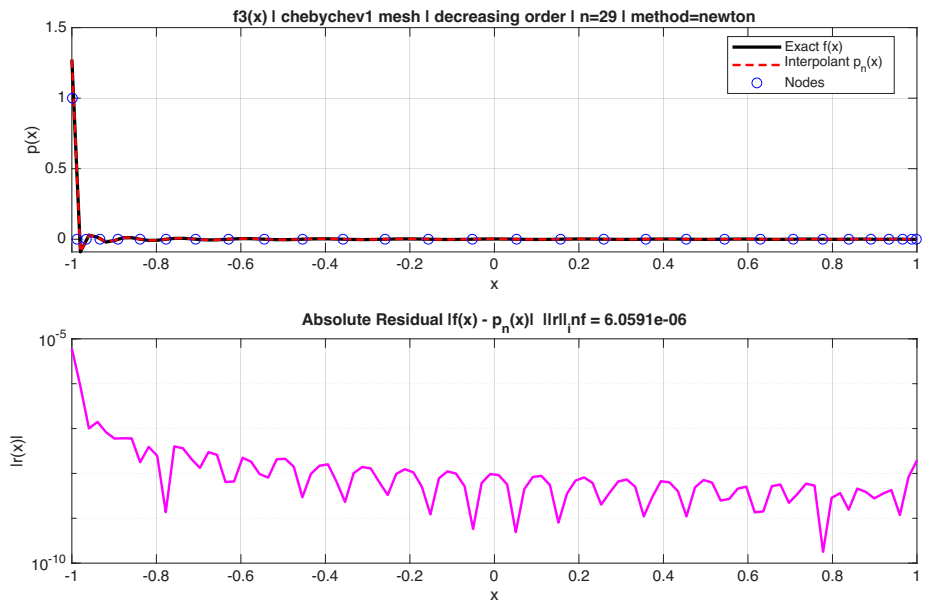

Figure 11: Task 4: Interpolation and Residual Plot

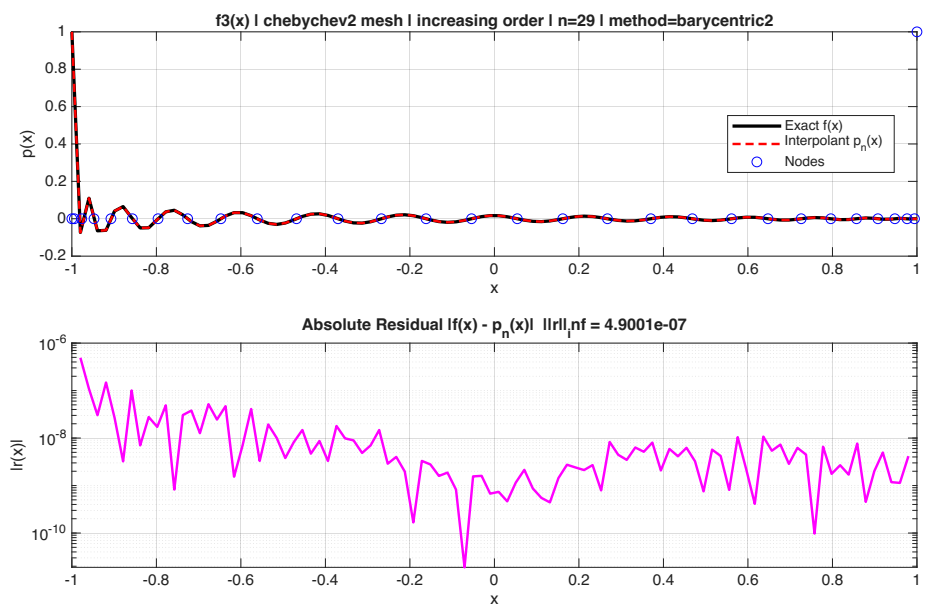

Figure 12: Task 3: Interpolation and Residual Plot

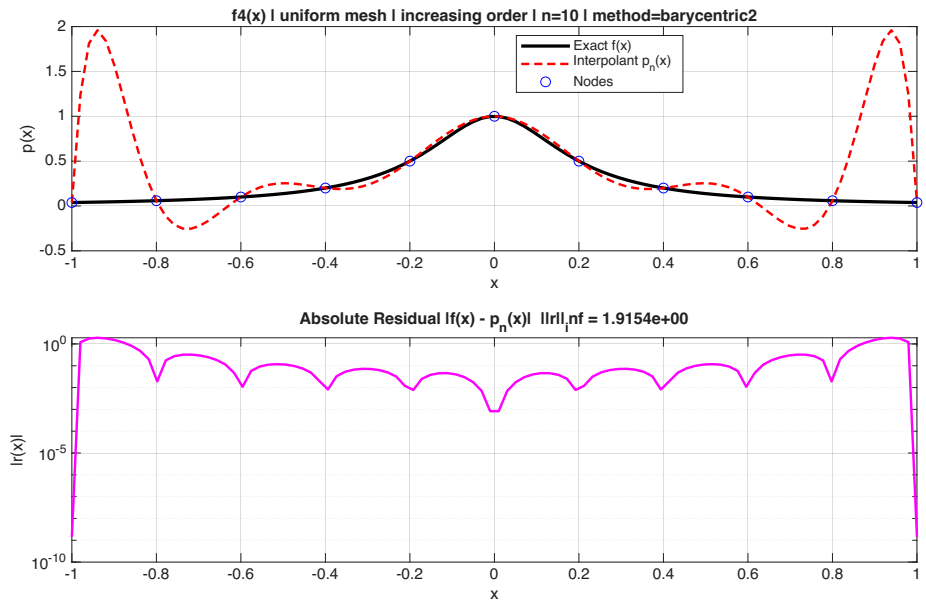

Figure 13: Task 5: Interpolation and Residual Plot

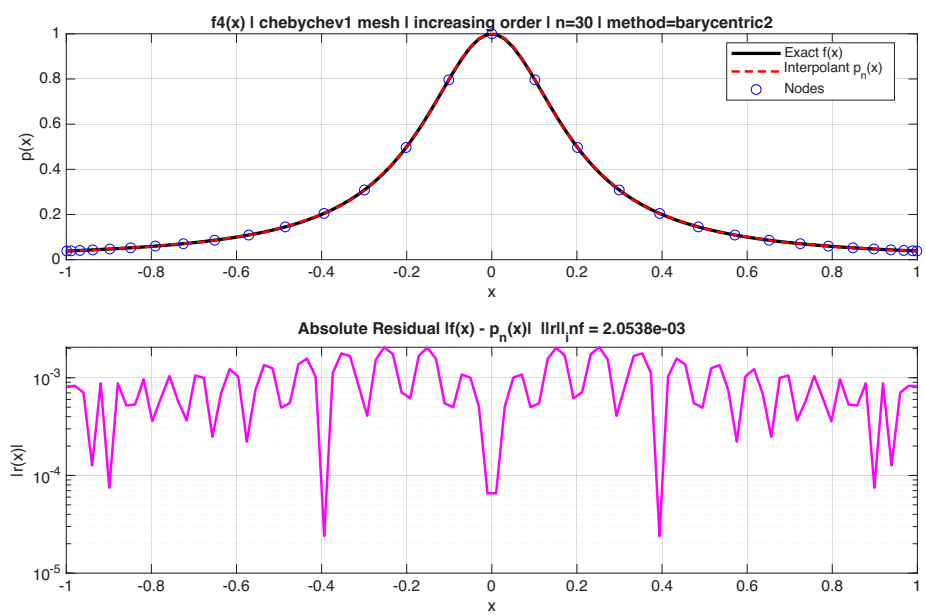

Figure 14: Task 5: Interpolation and Residual Plot

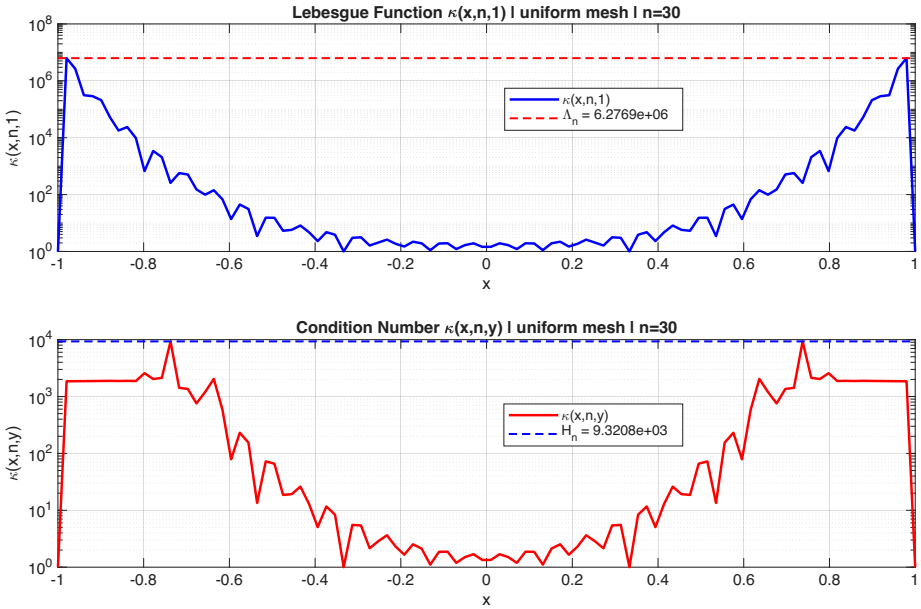

Figure 15: Task 5: Conditioning Plot

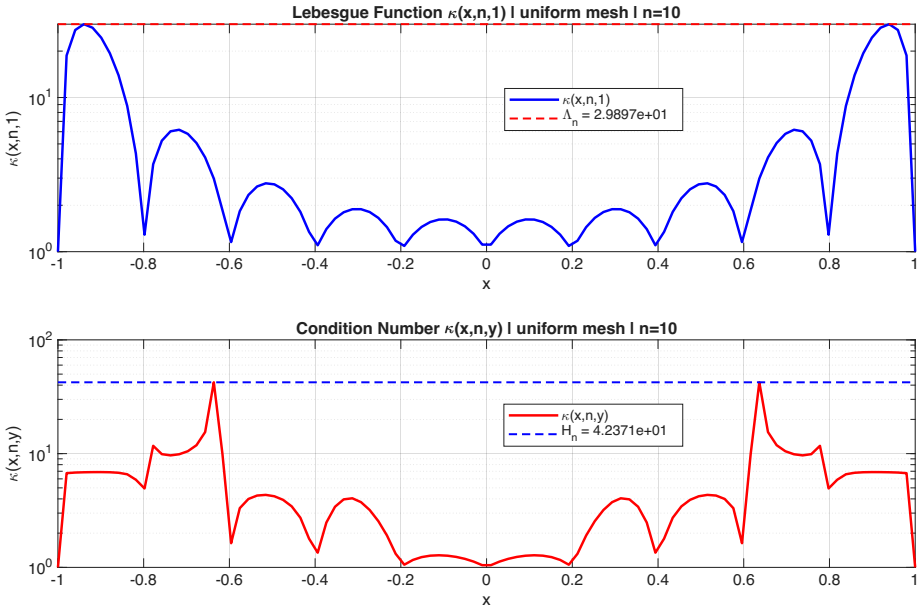

Figure 16: Task 5: Conditioning Plot

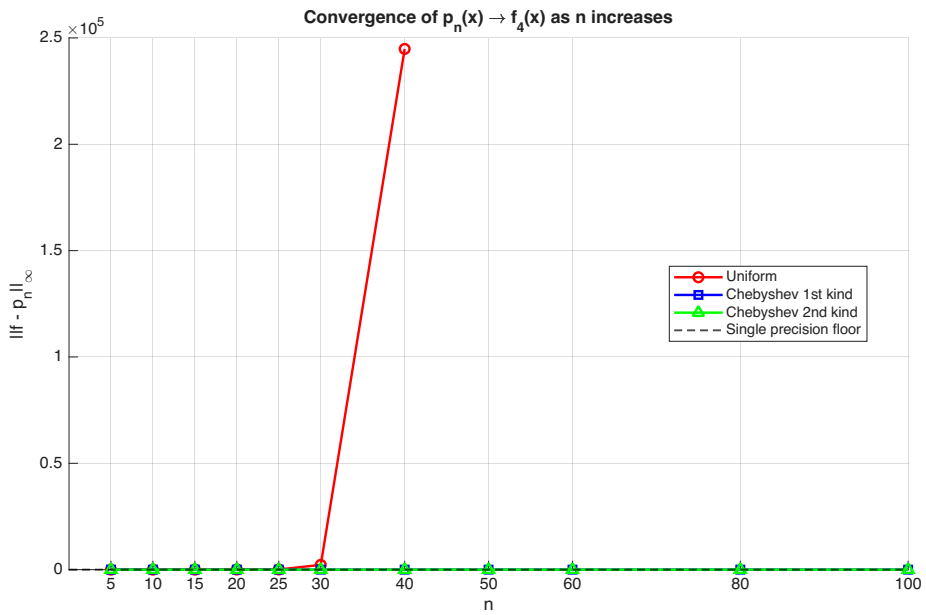

Figure 17: Task 5: Convergence Plot